Documento de Diseño – FitnessAhora

Contenido

Documento de Diseño - FitnessAhora 1

1. Introducción 2
2. Objetivo del Diseño 2
3. Alcance del Sistema 2
4. Arquitectura del Sistema 3
5. Diseño de Interfaces (UI/UX) 3
6. Paleta de Colores (Tema Oscuro) 5
7. Estructura del Sistema (Frontend) 6
8. Diseño de Base de Datos 6
9. Diccionario de Datos (Principales Tablas): 7

   Tabla: Usuarios 7

   Tabla: Ejercicios 8

10. Diseño de API (REST) 9
11. Consideraciones Técnicas 9
12. Escalabilidad y Futuras Mejoras 10
13. Conclusión 11

1. Introducción

FitnessAhora es una aplicación móvil diseñada para centralizar la gestión física del usuario, integrando módulos de entrenamiento estructurado, nutrición adaptada por somatotipo y suplementación. El sistema está concebido para operar bajo una arquitectura moderna, garantizando persistencia de datos relacionales y una interfaz nativa fluida.

Este documento establece las directrices técnicas, arquitectónicas y visuales para la fase de construcción. Servirá como mapa de ruta para la implementación del código fuente, el diseño de la base de datos y la configuración del servidor, asegurando que los requerimientos de la fase de análisis se traduzcan en componentes de software tangibles.

2. Objetivo del Diseño

Traducir los requisitos funcionales (como la clasificación por nivel de flexiones y somatotipo) en un esquema de datos relacional y una arquitectura cliente-servidor robusta. Se busca establecer una base de código escalable que permita el desarrollo ágil de la interfaz móvil, la exposición segura de la lógica de negocio mediante una API REST y la optimización del rendimiento en dispositivos móviles.

3. Alcance del Sistema

Cobertura del diseño:

- Arquitectura integral (Frontend móvil y Backend).
- Estructura de la base de datos relacional y flujos de información.
- Diseño lógico de interfaces de usuario (UI) y experiencia (UX).
- Definición de endpoints para la comunicación de datos.

Fuera del alcance (Fase inicial):

- Integración con hardware biométrico (smartwatches, básculas inteligentes).
- Sistemas de pago integrados.
- Videollamadas o streaming en vivo.

4. **Arquitectura del Sistema**

Propuesta: Arquitectura Cliente-Servidor basada en el modelo RESTful.

Tecnologías:

- **Frontend Móvil:** Ionic React (con Vite) + Capacitor para acceso nativo a hardware.
- **Backend / API:** Node.js con Express.
- **Base de Datos:** PostgreSQL (Stack PERN).

Diagrama Lógico (Flujo):

1. **Dispositivo Móvil (Cliente):** La app Ionic React renderiza la UI. Almacena en Capacitor Storage credenciales y datos offline.
2. **Red (Puente):** Llamadas HTTP estandarizadas (Axios/Fetch) con headers de autorización (JWT).
3. **Servidor Express (API Gateway):** Recibe la petición, pasa por middlewares (CORS, validación JWT, manejo de errores).
4. **Controladores/Servicios:** Ejecutan la lógica de negocio (ej. calcular dieta basada en somatotipo).
5. **Capa de Datos:** Se interactúa con PostgreSQL (mediante un ORM como Prisma o Sequelize) para leer/escribir.

5. **Diseño de Interfaces (UI/UX)**

Las interfaces seguirán un enfoque de componentes modulares. El contenido informativo se estructurará mediante tarjetas (cards) limpias.

A. **Pantalla de Perfilamiento (Onboarding)**

- **Propósito:** Capturar métricas biométricas iniciales.
- **Flujo:** Registro -> Seleccionar Somatotipo -> Ingresar test de flexiones -> Seleccionar objetivo.

B. Dashboard (Inicio)

- **Propósito:** Resumen diario del usuario.
- **Componentes Principales (Tarjetas de Resumen):**
  - 🔥 **Objetivo Actual:** Volumen / Definición / Mantenimiento.
  - 📅 **Entrenamiento de Hoy:** Día de Piernas (Ir a rutina).
  - 💧 **Recordatorio:** Hidratación y consumo de creatina.

C. Módulo de Entrenamiento (Detalle de Ejercicio)

- **Propósito:** Guiar la ejecución técnica.
- **Componentes Principales (Tarjetas de Ejecución):**
  - 💪 **Músculo Objetivo:** Pectoral mayor y tríceps.
  - ⚙️ **Modalidad:** Peso corporal / Pesas.
  - 🔄 **Volumen Prescrito:** 4 Series x 10 Repeticiones.
  - ⚠️ **Alerta Técnica:** Mantener los codos a 45 grados.
  - ▶ **Referencia:** Animación visual del movimiento en bucle.

D. Módulo de Alimentación

- **Propósito:** Visualizar la distribución de macronutrientes.
- **Flujo:** Ver resumen calórico diario -> Desplegar lista de alimentos por comida (Desayuno, Almuerzo, Cena).

6. **Paleta de Colores (Tema Oscuro)**

Diseño enfocado en reducir la fatiga visual durante entrenamientos en entornos con poca luz (gimnasios) y transmitir energía tecnológica.

Variable	**Color HEX**	**Uso / Justificación**
Background Base	#121212	Fondo principal. Ahorra batería en pantallas OLED.
Superficie (Cards)	#1E1E1E	Contenedores y tarjetas para crear contraste de elevación.
Color Primario	#00E676	Verde neón. Botones principales y CTAs. Transmite vitalidad.
Color Secundario	#2979FF	Azul eléctrico. Enlaces y acentos de navegación inferior.
Texto Principal	#FFFFFF	Alto contraste para legibilidad de instrucciones.
Texto Secundario	#B0B0B0	Descripciones largas o notas al pie.
Estado de Éxito	#00C853	Serie completada, meta de calorías alcanzada.
Estado de Error	#FF1744	Alertas de sistema o validaciones de formulario.

7. Estructura del Sistema (Frontend)

El proyecto Ionic React se estructurará mediante Vite para empaquetado rápido.

Árbol de Directorios:

- /src/assets/: Imágenes locales, iconos y animaciones Lottie/WebP.
- /src/components/: Componentes UI reutilizables (Cards, Botones, Inputs).
- /src/pages/: Vistas completas ruteadas (Dashboard, Login, EjercicioDetalle).
- /src/hooks/: Lógica personalizada (ej. useAuth, useRutinas).
- /src/services/: Funciones de llamada a la API (Axios instance).
- /src/context/: Manejadores de estado global (Usuario, Tema).

8. Diseño de Base de Datos

Modelo Conceptual:

Un Usuario tiene un nivel físico y un somatotipo. El usuario consulta Rutinas, las cuales están compuestas por múltiples Ejercicios. Cada Ejercicio se enfoca en un grupo muscular específico. Adicionalmente, el sistema tiene una biblioteca de Alimentos y Suplementos.

9. Diccionario de Datos (Principales Tablas):

Tabla: Usuarios

Campo	Tipo	Restricciones	Descripción
id_usuario	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()	Identificador único del usuario.
nombre	VARCHAR(100)	NOT NULL	Nombre de pila del usuario.
somatotipo	VARCHAR(20)	NOT NULL, CHECK (somatotipo IN ('ECTOMORFO','MESOMORFO','ENDOMORFO'))	Tipo de cuerpo del usuario.
max_flexiones	INT	NOT NULL, CHECK (max_flexiones >= 0)	Número máximo de flexiones realizadas (test físico).
nivel	VARCHAR(20)	NOT NULL, CHECK (nivel IN ('PRINCIPIANTE','INTERMEDIO','AVANZADO'))	Nivel calculado según desempeño físico.
objetivo	VARCHAR(50)	NOT NULL, CHECK (objetivo IN ('VOLUMEN','DEFINICION','MANTENIMIENTO'))	Objetivo fitness del usuario.
fecha_creacion	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha de registro del usuario.

Tabla: Ejercicios

Campo	Tipo	Restricciones	Descripción
id_ejercicio	UUID	PK, NOT NULL, DEFAULT uuid_generate_v4()	Identificador único del ejercicio.
id_grupo_muscular	UUID	FK, NOT NULL	Referencia a la tabla GrupoMuscular.
nombre	VARCHAR(100)	NOT NULL	Nombre del ejercicio.
equipamiento	VARCHAR(50)	NOT NULL, CHECK (equipamiento IN ('PESAS','BANDAS','CORPORAL'))	Tipo de equipamiento requerido.
nivel	VARCHAR(20)	NOT NULL, CHECK (nivel IN ('PRINCIPIANTE','INTERMEDIO','AVANZADO'))	Nivel de dificultad del ejercicio.
descripcion	TEXT	NOT NULL	Explicación detallada de la ejecución del ejercicio.
consejos	TEXT	NULL	Recomendaciones para una correcta ejecución.
parte_cuerpo	VARCHAR(100)	NOT NULL	Parte específica del cuerpo que se trabaja.
url_asset	VARCHAR(255)	NULL	Ruta a imagen/animación (ej: .webp optimizado).
creado_en	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha de creación del registro.

10. Diseño de API (REST)

Endpoint de Autenticación

- **URL:** /api/v1/auth/login
- **Método:** POST
- **Request:** { "email": "user@correo.com", "password": "***" }
- **Response:** { "token": "jwt_string", "user": { "somatotipo": "Mesomorfo" } }

Endpoint de Catálogo de Ejercicios

- **URL:** /api/v1/ejercicios?equipamiento=bandas&grupo=pecho
- **Método:** GET
- **Response:** Arreglo JSON de objetos con la estructura de la tabla Ejercicios.

11. Consideraciones Técnicas

- **Manejo de Estado:** Se utilizará React Context para el estado de autenticación y preferencias del usuario. Para los datos volátiles del servidor (rutinas, dietas), se recomienda React Query o control manual mediante custom hooks.
- **Manejo de Errores:** Interceptores de Axios globales. Si el token expira (401), se redirige silenciosamente al Login.
- **Validaciones:** Validar todos los payloads en el backend (Express) usando librerías como Joi o Zod antes de tocar PostgreSQL.

12. Escalabilidad y Futuras Mejoras

- **Modo Offline:** Implementar Service Workers (PWA) o SQLite local vía Capacitor para consultar rutinas descargadas en zonas sin cobertura.
- **Geolocalización:** A futuro, sugerir gimnasios o tiendas de suplementación locales (por ejemplo, en el área de Neiva) mediante el GPS del dispositivo.
- **Motor de Recomendación:** Migrar la lógica condicional simple a un modelo ligero de Machine Learning que ajuste las repeticiones semanales basado en el registro de progreso del usuario.

13. Conclusión

El diseño planteado para FitnessAhora capitaliza un ecosistema maduro y de alto rendimiento. Al estructurar la base de datos para relacionar las capacidades biométricas del usuario con las entidades físicas (ejercicios y alimentación), el sistema adquiere un carácter verdaderamente personalizado. La separación clara entre la interfaz móvil y los servicios backend garantiza un código limpio, auditable y preparado para iteraciones profesionales.